Week7 메모리 키워드 학습 가이드

재혁님이 **CSAPP 9.9 Dynamic Memory Allocation**을 읽고 나서, 이번 주 이슈 키워드들을 이어서 공부할 수 있도록 만든 정리본입니다. 이미지에 나온 이슈 순서를 그대로 반영했고, 각 키워드가 9.9와 어떻게 연결되는지도 같이 적었습니다.

가상메모리 페이징 힙 sbrk malloc/free 단편화 할당 정책 free list system call DMA

먼저 알고 가면 좋은 큰 흐름

가상메모리는 프로세스가 쓰는 메모리의 큰 틀이고, **힙**은 그 안에서 동적 할당에 쓰는 영역입니다. **malloc/free**는 힙을 관리하는 함수들이고, 그 안에서 **단편화**, **할당 정책**, **free list** 같은 개념이 등장합니다. 운영체제와 연결되는 아래쪽에는 **시스템콜**과 **sbrk/mmap**이 있고, 하드웨어 쪽에는 **DMA** 같은 개념이 이어집니다.

1. 가상메모리 & 페이징

정의: 가상메모리는 각 프로세스가 자신만의 큰 메모리를 가진 것처럼 보이게 하는 추상화입니다. 페이징은 이 가상 주소 공간을 페이지 단위로 나눠서 실제 물리 메모리와 연결하는 방식입니다.

- 프로세스는 연속된 큰 주소 공간을 가진 것처럼 보입니다.
- 실제로는 페이지 단위로 나뉘어 물리 메모리에 매핑됩니다.
- 필요한 페이지만 메모리에 올리는 demand paging이 가능합니다.

9.9와 연결

9.9에서 allocator가 관리하는 **heap도 결국 가상메모리의 일부**입니다. 즉, malloc이 주는 메모리는 그냥 "RAM 일부"가 아니라, 가상 주소 공간 안의 힙 영역에서 관리되는 block입니다.

2. 동적 메모리 할당 (힙 & sbrk & malloc & free)

정의: 실행 중 필요한 크기만큼 메모리를 요청하고 반납하는 방식을 동적 메모리 할당이라고 합니다.

키워드	핵심 의미
heap	동적 메모리 할당에 사용하는 메모리 영역
sbrk	힙 끝을 늘려 추가 공간을 확보하는 오래된 방식
malloc	요청 크기에 맞는 block을 찾아서 반환
free	사용이 끝난 block을 다시 free 상태로 돌림

왜 필요한가

프로그램은 실행 전에 모든 데이터 크기를 알 수 없기 때문에, 필요한 순간에 필요한 만큼 메모리를 잡을 수 있어야 합니다. 연결리스트, 트리, 가변 크기 버퍼가 대표적인 예입니다.

9.9와 연결

9.9의 중심 자체가 바로 이 부분입니다. allocator는 힙을 block 단위로 관리하고, block이 부족하면 `sbrk`나 `mmap` 같은 방식으로 더 큰 공간을 받아옵니다.

3. 메모리 단편화

정의: 메모리에 빈 공간은 남아 있지만, 그 공간이 비효율적으로 나뉘어 있어서 원하는 방식으로 쓰기 어려운 상태입니다.

내부 단편화

할당된 block이 payload보다 커서 block 안에 남는 낭비입니다. 정렬이나 최소 block 크기 때문에 자주 생깁니다.

외부 단편화

전체 free 공간은 충분하지만, 큰 연속 block이 없어 요청을 못 처리하는 상태입니다.

9.9와 연결

9.9에서 **split**은 내부 단편화와, **coalesce**는 외부 단편화와 깊게 연결됩니다. allocator 설계는 결국 단편화를 얼마나 잘 줄이느냐의 문제이기도 합니다.

4. 메모리 할당 정책 (first fit & next fit & best fit)

정의: free block들 중 어떤 block을 선택해서 할당할지 정하는 규칙입니다.

정책	아이디어	느낌
first fit	처음 만나는 충분히 큰 free block 사용	단순하고 구현 쉬움
next fit	직전 탐색 지점부터 이어서 찾기	앞부분 반복 탐색을 줄이려는 방식
best fit	가장 딱 맞는 block 찾기	탐색 비용이 커질 수 있음

중요한 감각

어떤 정책이 무조건 최고라고 보긴 어렵습니다. 속도, 단편화, 구현 복잡도가 서로 trade-off 관계에 있습니다.

9.9와 연결

9.9의 implicit free list 부분을 읽으면 allocator가 free block을 어떻게 찾는지가 나오는데, 바로 이 부분이 할당 정책입니다.

5. demand-zero memory

정의: 운영체제가 새 메모리 페이지를 프로세스에 줄 때, 처음에는 모두 0으로 채워진 것처럼 보이게 하는 메모리입니다.

- 실제로는 필요한 시점에 페이지가 준비되는 경우가 많습니다.
- 새로 받은 메모리가 이전 프로세스의 쓰레기 값으로 보이면 안 되기 때문에 안전상 중요합니다.
- 프로그램 입장에서는 "새로운 깨끗한 메모리"처럼 느껴집니다.

9.9와 연결

9.9 앞부분에서도 heap을 demand-zero memory라고 설명합니다. 즉, 새로 늘어난 힙 공간은 처음엔 0으로 초기화된 깨끗한 영역이라는 감각을 가져가면 됩니다.

6. implicit free list & explicit free list

정의: free block들을 어떤 자료구조로 관리할지에 대한 방식입니다.

방식	설명	장단점
implicit free list	모든 block을 순회하면서 header의 size 정보를 이용해 free block을 찾음	단순하지만 탐색이 느릴 수 있음
explicit free list	free block들만 포인터로 연결해 별도 리스트처럼 관리	free block만 보니 더 효율적이지만 구현이 복잡

9.9와 연결

9.9는 먼저 implicit free list로 allocator 개념을 익히게 하고, 그 다음 explicit free list 같은 더 효율적인 구조로 확장합니다. malloc-lab도 보통 여기서 점수를 더 올리는 방향으로 갑니다.

7. 시스템콜

정의: 사용자 프로그램이 운영체제 커널에게 특정 작업을 요청하는 공식 통로입니다.

- 파일 열기, 읽기, 쓰기, 프로세스 생성, 메모리 요청 같은 작업이 여기에 포함됩니다.
- 일반 함수 호출과 다르게 커널 모드와 사용자 모드의 경계를 넘는 요청입니다.
- C 라이브러리 함수 뒤에서 실제 시스템콜이 사용되는 경우가 많습니다.

9.9와 연결

allocator는 모든 메모리를 스스로 만들어내는 게 아니라, 필요하면 결국 운영체제에게 더 큰 메모리 영역을 요청해야 합니다. 그 아래쪽에는 `sbrk`, `mmap` 같은 시스템 수준 인터페이스가 있습니다.

8. DMA

정의: DMA(Direct Memory Access)는 CPU가 일일이 복사하지 않아도, 장치가 메모리와 직접 데이터를 주고받을 수 있게 하는 방식입니다.

- 디스크, 네트워크 장치, 각종 입출력 장치에서 많이 사용됩니다.
- CPU가 매 바이트마다 개입하지 않아도 되므로 효율이 좋아집니다.
- 운영체제, 메모리, 하드웨어가 함께 얽힌 개념입니다.

이번 주 관점에서 보면

DMA는 malloc-lab 구현에 직접 쓰는 개념은 아니지만, 메모리가 CPU만의 공간이 아니라 장치와도 연결되는 자원이라는 감각을 넓혀줍니다. 시스템 프로그래밍 관점에서 메모리를 더 넓게 이해하는 데 도움이 됩니다.

9. 재혁님 추천 학습 순서

1. **1순위**: 가상메모리 & 페이징
2. **2순위**: 동적 메모리 할당 (힙, sbrk, malloc, free)
3. **3순위**: 메모리 단편화
4. **4순위**: 메모리 할당 정책
5. **5순위**: implicit free list & explicit free list
6. **6순위**: demand-zero memory
7. **7순위**: 시스템콜
8. **8순위**: DMA

이 순서를 추천하는 이유

앞쪽은 allocator와 직접 연결되는 개념이고, 뒤쪽은 운영체제와 시스템 전체 감각을 넓혀주는 개념이기 때문입니다. 재혁님은 이번 주에 **malloc-lab**과 **9.9**가 핵심이니, 앞의 다섯 개를 우선 제대로 잡는 게 가장 중요합니다.

10. 마지막 한 장 요약

- 가상메모리는 큰 틀, 힙은 동적 할당 영역
- malloc/free는 힙의 block을 관리하는 도구
- 단편화는 allocator가 풀어야 할 대표 문제
- 할당 정책은 어떤 free block을 줄지 정하는 규칙
- implicit/explicit free list는 free block 관리 방식의 차이
- demand-zero memory와 시스템콜은 운영체제와의 연결 지점
- DMA는 메모리를 시스템 전체 관점에서 이해하게 해주는 개념

읽을 때는 각 키워드를 따로 외우기보다, "이 개념이 heap allocator 안에서 어떤 역할을 하는가?"를 계속 연결해서 보는 것이 훨씬 도움이 됩니다.